

USER MANAGEMENT SYSTEM API

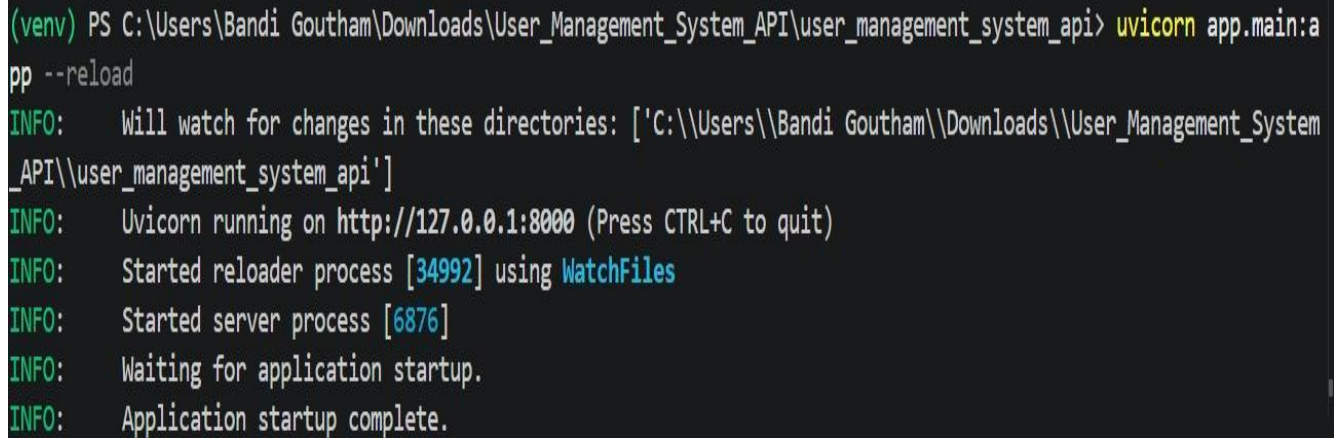
Testing & Results Report

Technology Stack: Python | FastAPI | MySQL | SQLAlchemy ORM | JWT Authentication | Pydantic |
Swagger UI

This report documents the implementation and testing of the User Management System API. The screenshots demonstrate user registration, authentication, JWT authorization, profile CRUD operations, MySQL persistence, validation, and final server responses.

1. Project Setup and Server Running

The FastAPI project is opened in VS Code with the application modules, virtual environment, environment file, requirements file, and Git ignore configuration. Uvicorn is running successfully on the local development server.

A terminal window with a dark background and green text. The prompt is '(venv) PS C:\Users\Bandi Goutham\Downloads\User_Management_System_API\user_management_system_api>'. The command 'uvicorn app.main:app --reload' has been entered. The output shows several INFO messages: 'Will watch for changes in these directories: [\'C:\\Users\\Bandi Goutham\\Downloads\\User_Management_System_API\\user_management_system_api\']', 'Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)', 'Started reloader process [34992] using WatchFiles', 'Started server process [6876]', 'Waiting for application startup.', and 'Application startup complete.'

```
(venv) PS C:\Users\Bandi Goutham\Downloads\User_Management_System_API\user_management_system_api> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Bandi Goutham\\Downloads\\User_Management_System_API\\user_management_system_api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [34992] using WatchFiles
INFO: Started server process [6876]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 1: Project Setup and Server Running

2. Swagger UI - API Endpoints

Swagger UI displays the complete User Management System API, including registration, login, profile creation, own-profile retrieval, user listing, profile update, and profile deletion endpoints.

User Management System API 1.0.0 OAS 3.1

/openapi.json

Beginner FastAPI + MySQL + SQLAlchemy + JWT authentication project.

Authorize 

Authentication

POST /auth/register Register a new user  

POST /auth/login Login and receive a JWT access token 

Users

PUT /users/profile Update the logged-in user's profile  

POST /users/profile Create the logged-in user's profile  

DELETE /users/profile Delete the logged-in user's profile  

GET /users/me Get the logged-in user's profile  

GET /users List registered users  

Health

GET / Root 

GET /health Health  

Schemas

HTTPValidationError > Expand all [object](#)

LoginRequest > Expand all [object](#)

ProfileCreate > Expand all [object](#)

ProfileResponse > Expand all [object](#)

ProfileUpdate > Expand all [object](#)

RegisterRequest > Expand all [object](#)

TokenResponse > Expand all [object](#)

TokenResponse > Expand all [object](#)

UserResponse > Expand all [object](#)

ValidationError > Expand all [object](#)

Figure 2: Swagger UI - API Endpoints

3. User Registration - Successful

The registration endpoint was tested successfully. The API returned HTTP 201 Created and returned the newly registered user's ID, name, and email.

Code	Details
201	Response body <pre>{ "id": 2, "name": "RAKESH", "email": "rakesh@gmail.com", "created_at": "2026-09-29T15:52:14", "profile": { "phone": "1234567890", "address": "hyd", "date_of_birth": "2026-09-29", "bio": "hello" } }</pre>  Download

Figure 3: User Registration - Successful

4. Duplicate Registration Validation

The registration endpoint correctly rejects an already registered email address and returns HTTP 400 Bad Request with the message 'Email already registered'.

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "rakesh",
    "email": "rakesh@gmail.com",
    "password": "raki@64190"
  }'
```

Request URL

http://127.0.0.1:8000/auth/register

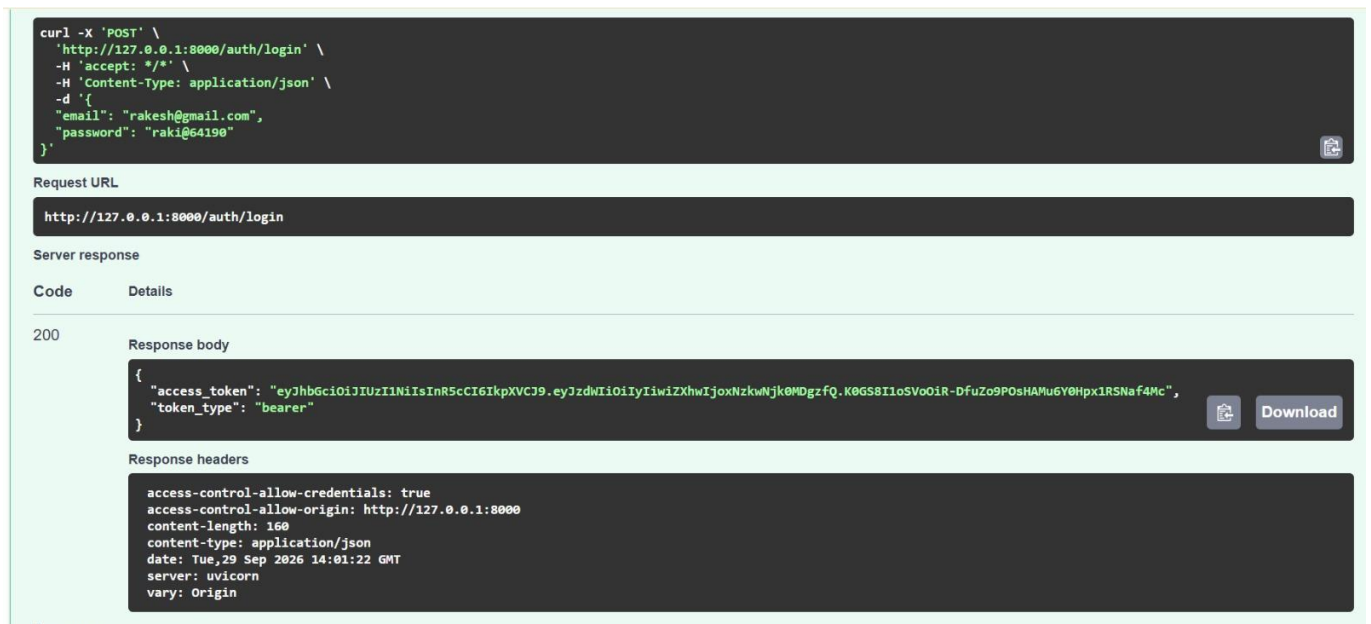
Server response

Code	Details
409 Undocumented	Error: Conflict Response body <pre>{ "detail": "Email is already registered" }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 40 content-type: application/json date: Tue, 29 Sep 2026 14:02:09 GMT server: uvicorn vary: Origin</pre>

Figure 4: Duplicate Registration Validation

5. User Login and JWT Token

The login endpoint accepts the registered email and password. A successful login returns HTTP 200 OK together with an access token and bearer token type.



The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://127.0.0.1:8000/auth/login`
 - Headers:
 - `accept: */*`
 - `Content-Type: application/json`
 - Body (JSON):

```
{  "email": "rakesh@gmail.com",  "password": "raki@64190"}
```
- Response:**
 - Status: 200
 - Body (JSON):

```
{  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiaXNjaXkiOiJ0NDgzfQ.K0GS8I1oSVo0iR-DfuZo9POsHAMu6Y0Hpx1RSnaf4Mc",  "token_type": "bearer"}
```
 - Headers:

```
access-control-allow-credentials: trueaccess-control-allow-origin: http://127.0.0.1:8000content-length: 160content-type: application/jsondate: Tue, 29 Sep 2026 14:01:22 GMTserver: uvicornvary: Origin
```

Figure 5: User Login and JWT Token

6. JWT Authorization

Swagger authorization is enabled using OAuth2 Password Bearer authentication. This allows protected endpoints to receive the JWT bearer token.

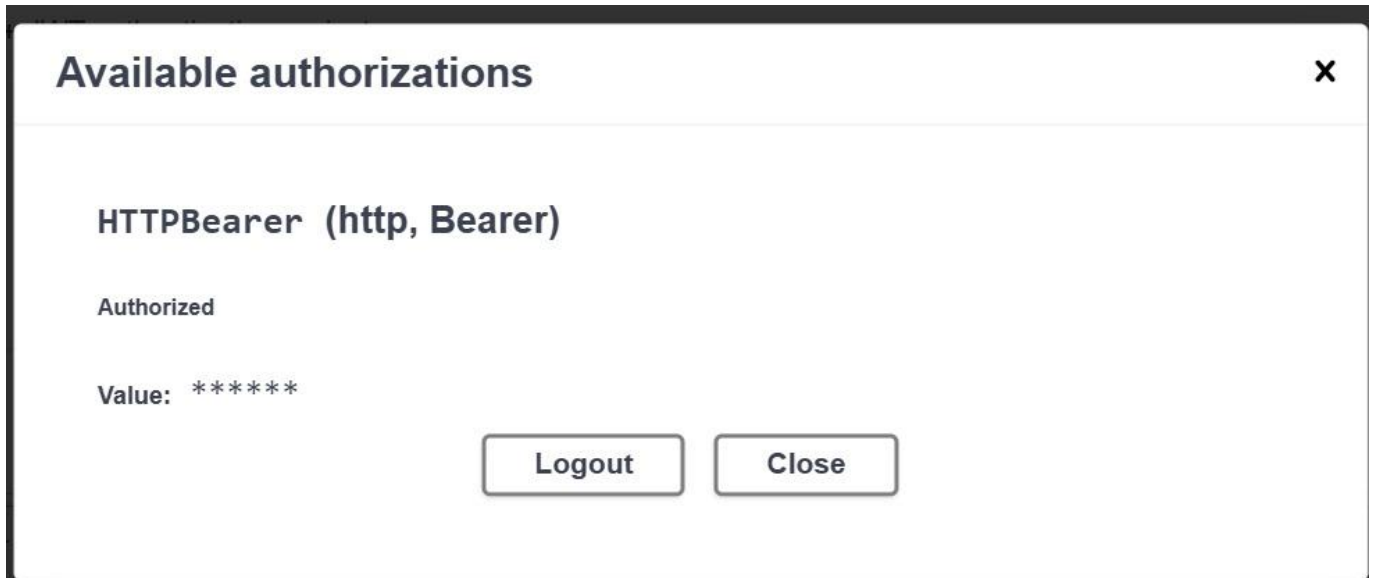


Figure 6: JWT Authorization

7. Create Profile

The authenticated user creates a profile containing phone number, address, date of birth, and bio. The API returns HTTP 201 Created and stores the profile with the corresponding user ID.

Request URL	
http://127.0.0.1:8000/users/profile	
Server response	
Code	Details
201	Response body <pre>{ "id": 2, "name": "RAKESH", "email": "rakesh@gmail.com", "created_at": "2026-09-29T15:52:14", "profile": { "phone": "7207255820", "address": "kodur", "date_of_birth": "2026-09-29", "bio": "Don" } }</pre>  Download Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 178 content-type: application/json date: Tue, 29 Sep 2026 14:06:50 GMT server: uvicorn vary: Origin</pre>

Figure 7: Create Profile

8. Get Own Profile

The protected GET `/users/me` endpoint retrieves the authenticated user's profile successfully and returns HTTP 200 OK.

```
curl -X 'GET' \
  'http://127.0.0.1:8000/users/me' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiaXhwIjoxNzkwNjk0MDgzfQ.K0GS8IloSvoOIR-DfuZo9POsHAMu6Y0HpX1RSNaF4Mc'
```

Request URL

http://127.0.0.1:8000/users/me

Server response

CodeDetails

200

Response body

```
{
  "id": 2,
  "name": "RAKESH",
  "email": "rakesh@gmail.com",
  "created_at": "2026-09-29T15:52:14",
  "profile": {
    "phone": "7207255820",
    "address": "kodur",
    "date_of_birth": "2026-09-29",
    "bio": "Don"
  }
}
```

Download

Response headers

```
content-length: 178
content-type: application/json
date: Tue, 29 Sep 2026 14:00:00 GMT
server: uvicorn
vary: Origin
```

Figure 8: Get Own Profile

9. List Users

The protected GET /users endpoint returns the registered users. The response confirms that the user-listing functionality is working successfully.

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "name": "Goutham", "email": "goutham@gmail.com", "created_at": "2026-09-29T14:38:41", "profile": { "phone": "1234567890", "address": "hyd", "date_of_birth": "2026-09-29", "bio": "hello" } }, { "id": 2, "name": "RAKESH", "email": "rakesh@gmail.com", "created_at": "2026-09-29T15:52:14", "profile": { "phone": "7207255820", "address": "kodur", "date_of_birth": "2026-09-29", "bio": "Don" } }]</pre>  Download

Figure 9: List Users

10. Update Profile

The PUT /users/profile endpoint updates the profile successfully. The response returns HTTP 200 OK and shows the updated profile data.



The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: PUT
 - URL: http://127.0.0.1:8000/users/profile
 - Headers:
 - Accept: */*
 - Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiaXNja3k0MDgzfQ.K0GS8IIoSVoOIR-DfuZo9POsHAMu6V0Hpx1RSNaf4Mc
 - Content-Type: application/json
 - Body (JSON):

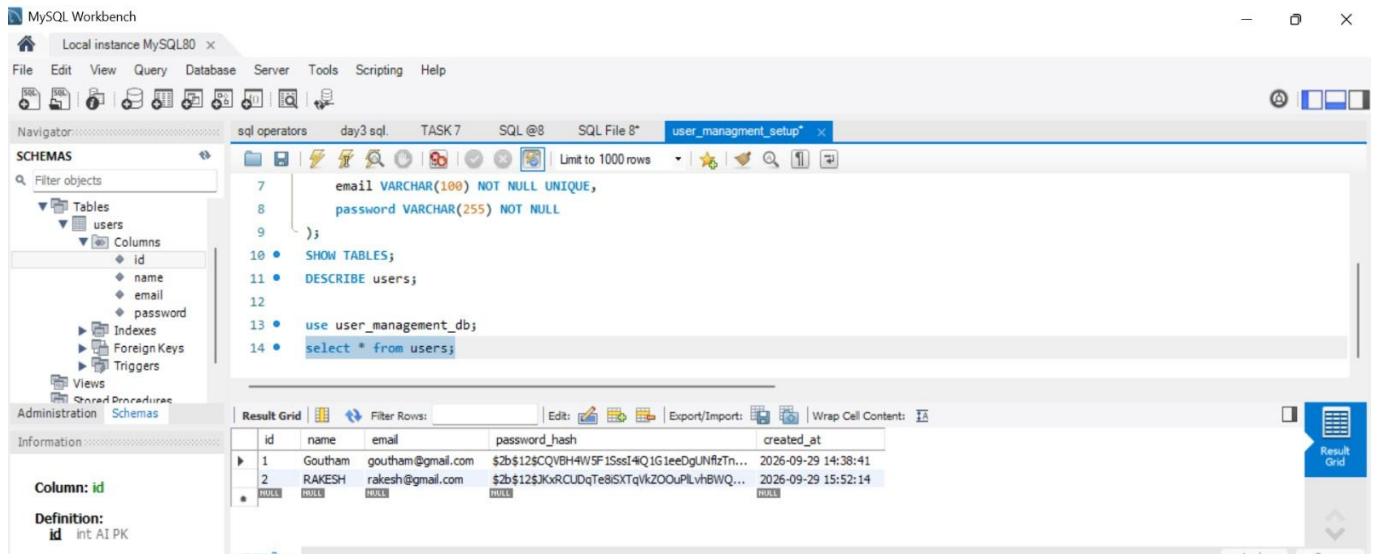
```
{  "phone": "123",  "address": "kochi",  "date_of_birth": "2026-09-29",  "bio": "updated"}
```
- Response:**
 - Status: 200
 - Body (JSON):

```
{  "id": 2,  "name": "RAKESH",  "email": "rakesh@gmail.com",  "created_at": "2026-09-29T15:52:14",  "profile": {    "phone": "123",    "address": "kochi",    "date_of_birth": "2026-09-29",    "bio": "updated"  }}
```

Figure 10: Update Profile

11. MySQL Users Table Verification

MySQL Workbench confirms that registered users are stored in the users table. Passwords are stored as hashed values rather than plain text.



The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the 'users' table with columns: id, name, email, password, and created_at. The 'password' column is highlighted, showing its definition: 'id int AI PK'. The main editor window shows the SQL script for creating the 'users' table and querying it. The 'Result Grid' at the bottom displays the data for the 'users' table.

```
7 email VARCHAR(100) NOT NULL UNIQUE,  
8 password VARCHAR(255) NOT NULL  
9 );  
10 SHOW TABLES;  
11 DESCRIBE users;  
12  
13 use user_management_db;  
14 select * from users;
```

id	name	email	password_hash	created_at
1	Goutham	goutham@gmail.com	\$2b\$12\$CQVBH4W9F1SssI4Q1G1eeDgUNftzTn...	2026-09-29 14:38:41
2	RAKESH	rakesh@gmail.com	\$2b\$12\$JkxRCLUDqTe8SXTqVikZOouPLvhBWQ...	2026-09-29 15:52:14
* NULL	NULL	NULL	NULL	NULL

Figure 11: MySQL Users Table Verification

12. Authentication Validation - No Token

A protected profile request without valid authentication returns HTTP 401 Unauthorized with the message 'Not authenticated', confirming access protection.



Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/users/me' \
-H 'accept: */*'
```

Request URL

<http://127.0.0.1:8000/users/me>

Server response

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "Missing Bearer token" }</pre> Download Response headers <pre>content-length: 33 content-type: application/json date: Tue, 29 Sep 2026 14:16:50 GMT server: uvicorn vary: Origin www-authenticate: Bearer</pre>

Figure 12: Authentication Validation - No Token

13. Invalid Login Validation

An invalid login attempt returns HTTP 401 Unauthorized with the message 'Invalid email or password', confirming credential validation.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "rakesh@gmail.com",
    "password": "raki@6419"
  }'
```

Request URL

http://127.0.0.1:8000/auth/login

Server response

Code	Details
401	Error: Unauthorized

Undocumented

Response body

```
{
  "detail": "Invalid email or password"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 38
content-type: application/json
date: Tue, 29 Sep 2026 14:17:49 GMT
server: uvicorn
```

Figure 13: Invalid Login Validation

14. Delete Profile

The authenticated DELETE /users/profile request succeeds with HTTP 200 OK and returns the message 'Profile deleted successfully'.

Responses

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/users/profile' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyIiwiaXhwIjoxNzkxNjgzNzUzZmEDXpovJIE-IzCj4fUYWgm-XSmPWOXT9Sk'
```

Request URL

```
http://127.0.0.1:8000/users/profile
```

Server response

Code	Details
204	Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-type: application/json date: Tue, 29 Sep 2026 11:10:52 GMT server: uvicorn vary: Origin</pre>

Responses

Code	Description	Links
204	Successful Response	No links

Figure 14: Delete Profile

15. Verify Deleted Profile

After deletion, GET /users/me returns HTTP 404 Not Found with the message 'Profile not found', confirming that the profile was removed.

Request URL

http://127.0.0.1:8000/users/profile

Server response

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Profile not found" }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 30 content-type: application/json date: Tue, 29 Sep 2026 14:21:30 GMT server: uvicorn vary: Origin</pre>

Responses

Code	Description	Links
204	Successful Response	No links

Figure 15: Verify Deleted Profile

16. Final Server and API Logs

The Uvicorn terminal log records the API tests, including successful 200/201 responses and expected 400/401/404 validation responses.

```
(venv) PS C:\Users\Bandi Goutham\Downloads\User_Management_System_API\user_management_system_api> uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Bandi Goutham\\Downloads\\User_Management_System_API\\user_management_system_api']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [34992] using WatchFiles
INFO: Started server process [6876]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:53765 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:11509 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:11509 - "GET /openapi.json HTTP/1.1" 200 OK
```

Figure 16: Final Server and API Logs

Conclusion

The screenshots demonstrate that the User Management System API supports registration, login, JWT-protected access, profile creation, profile retrieval, user listing, profile updating, and profile deletion. MySQL Workbench screenshots also verify database persistence, while the 400, 401, and 404 responses demonstrate validation and protected-route behavior.